POLYDUB

Ukah-Columba Chinaza Howard
howardukah@gmail.com
Adamu Adamu Sule
Kynad0h06@gmail.com

Guided by:
Divya Kavathiya
divya.kavathiya@marwadieducation.edu.in

ABSTRACT

Polydub is a mobile application that translates and dubs speech in real time, aiming to make videos and audio on Android phones free from language barriers like turning a French podcast into smooth, clear English on the fly. The app grabs system audio(MediaProjection,AudioPlaybackCapture) and slices it into short clips, using a lightweight voice activity detector that stops the moment the sound fades. It then taps the Vosk engine to handle speech recognition right on the device, picking up even a faint whisper. Google's ML Kit Translation models handle the recognized text offline, turning it into another language, and Android's Text-to-Speech then speaks it aloud in a clear, steady voice. Editors use timestamp matching, audio ducking, and time-stretching to keep the voice in step with the lips, so you hear each word as the mouth moves, while the subtitles roll across the bottom of the screen. Running as a foreground service with floating controls, it lets you keep dubbing in the background and jump in instantly like tapping a button that appears right on top of your video. They ran the experiments on OTT shows, university lectures, and podcasts, including one with the faint sound of rain in the background. The tool runs smoothly on mid-range devices, responding in a snap, and it works entirely on your machine. The model has a few drawbacks: it's large, works only on Android 10 or newer, and sometimes the lip-sync slips like a half-beat delay in a song.

1. INTRODUCTION

As digital communication rapidly spreads worldwide, the demand for multilingual accessibility in videos, podcasts, and other media platforms is increasing. Billions of people watch films, listen to podcasts, and scroll through feeds every day. However, most of this content is in only a handful of languages (English, Mandarin, and Spanish), excluding those who cannot fully understand it. Methods such as subtitles and hand-done dubbing attempt to bridge this gap, but they don't scale well and often break immersion (e.g., when your eyes are suddenly pulled away by a block of text). Subtitles pull your eyes off the action. Manual dubbing, wherein multiple pricey translators, voice actors and editors re-record every word, seldom captures natural timing or that slight lift of a voice that makes a moment feel real. Cloud-based translation tools such as Google Translate can handle text or speech inputs but not an entire video or livestream in real-time. Polydub set out to solve this by

building an Android application that can capture or system audio, extract text, translate it into a target language, and play perfectly timed dubbing with subtitles right on the device (think of a French film whispering English in your ear). Polydub works offline once you download the model, unlike other tools that require an internet connection. The system leverages Vosk (speech-to-text), ML Kit (translation), and Android TTS (audio synthesis) in a modular pipeline with overlay controls to create a fast way to make everyday media like a live news clip accessible to

1.2 Aim and Objective

This project seeks to develop an offline-first, AI-powered Android application that will provide real-time dubbing and subtitles for multimedia content. The objectives are:

i. Capture audio from the system playback and microphone in real-time.

ii. Perform accurate on-device speech recognition with Vosk.

iii. Use Google ML Kit offline models to translate the recognized text.

iv. Create synchronized dubbing via Android TTS.

v. Provide real-time captions overlay for videos with accessibility presets.

vi. Enable background dubbing with floating controls to allow multitasking.

vii. Improve performance with regard to system resources (low latency, CPU usage, and seamless synchronization).

1.3 Problem Specifications

Functional requirements:

i. Real-time audio capture (microphone + system audio).

ii. On-device STT using multiple languages.

iii. Low-latency translation in 50+ languages.

iv. Natural-voice TTS dubbing with volume ducking.

v. Subtitle rendering using overlay widget.

vi. Background dubbing using floating controls.

Non-functional requirements:

i. Latency below 500ms for seamless playback.

ii. Compatibility for Android 10+ handsets.

iii. Off-line mode for privacy and accessibility.

iv. Reliability vs. crashes and overheating.

v. Light and accessible interface.

Constraints:

i. Vosk model sizes (50MB–2GB).

ii. High CPU usage during simultaneous STT + TTS.

iii. Imperfect lip synchronization due to language length differences.

iv. Limited support on Android 10 devices.

1.4 Literature Review

As a prelude to the Project on PolyDub, the available research and technologies on the topic of speech translation and automatic

dubbing were studied. We studied several works, including those on prosody-aware speech-to-speech translation, multimodal TTS with lip synchronization, cross-lingual style transfer, and length-controlled machine translation. These papers provide useful information on the kind of issues like timing mismatches, naturalness, and expressive dubbing that our work addresses and the voids that PolyDub attempts to fill. The key contributions of these works and how they bear on our task can be summarized in the table that follows.

i. Marcello Federico et al. (2020), "From Speech-to-Speech Translation to Automatic Dubbing" This work specified a pipeline of ASR → MT → prosody-aware TTS, audio rendering, and mixing of noise. This process resembles Polydub's own real-time dubbing pipeline most closely when translating speech overlaid onto pre-existing audio.

ii. Bowen Hu et al. (2021), "Neural Dubber: Dubbing for Videos According to Scripts." The authors designed a multimodal TTS system capable of carrying out script alignment, preservation of speaker identity, and lip sync. This enhances dubbing's naturalness and temporal coherence and brings dubbing closer to Polydub's goals for immersive experience delivery.

iii. Zichang Li et al. (2023), "Joint Multi-scale Cross-lingual Speaking Style Transfer" It used bidirectional attention and multi-scale alignment for transference of speaking style across languages. This allows for expressive dubbing that Polydub can take advantage of for maintaining speaker tone and emotional expression in translations.

iv. Kai Wu et al. (2023), "VideoDubber: MT with Speech-Aware Length Control." This research introduced duration-controlled machine translation (MT) to improve alignment of speech duration with playing videos for addressing the issue of timing mismatches. Polydub addresses the similar challenge using lightweight time-stretching methods.

v. Harveen Singh Chadha et al. (2025), "Length Aware Speech Translation for Video Dubbing." This paper proposed length-aware phoneme-level beam search and multi-length decoding at the proper sync of the translated speech and the video timing. Polydub targets using the similar concepts to get better lip-sync accuracy.

vi. Dimitrios Floros et al. (2023), "Multilingual Video Dubbing: Tech Review & Challenges." This paper reviewed dubbing technologies and found key challenges including latency and sync issues. It points to the gaps that exist and most of them Polydub explicitly covers by going offline-first and mobile-first

1.5 Plan of the Work

We employed the stepwise procedure:

i. Requirement analysis and STT engine assessment (Vosk vs. Whisper).

ii. System pipeline architecture design.

iii. Audio capture implementation (MediaProjection + AudioPlaybackCapture).

iv. STT integration with Vosk.

v. Translation engine integration (ML Kit).

vi. TTS dubbing with audio ducking and sync.

vii. Overlay and subtitle design.

viii. Background dubbing implementation.

ix. Time-stretching and performance tuning.

x. Testing on OTT shows, lectures, and podcasts.

1.6 Materials/Tools Required

i. Hardware: Android 10+ smartphone (≥4GB RAM), development laptop (≥8GB RAM)

ii. Software: Android Studio, Kotlin, Gradle

iii. Libraries: Vosk STT, Firebase ML Kit Translation, Android TTS, MediaProjection API

iv. Sources: Research articles, user response, selected test media

2. ANALYSIS AND DESIGN METHODOLOGY

2.1 Observation Matrix

The field research pinpointed areas for enhancement: subtitle fatigue, poor lip sync, background music overwhelming speech, poor internet connectivity, and exposure concerns. The potential areas were on-device STT, translating offline, audio ducking, timestamp sync alignment, and floating overlays. The observation matrix defined Polydub's user-centric design.

2.2 Ideation Canvas

i. Personae: students, instructors, OTT viewers, deaf viewers.

ii. Pains: subtitle distraction, desyncing, privacy.

iii. Solutions: on-device STT, offline translation, overlay captions, background dubbing.

iv. Value proposition: "Instant, private, offline dubbing and captions for any media app."

v. Differentiators: system-level capture, hybrid dubbing+captions, privacy-first design.

2.3 Product Development Canvas

i. Purpose: to translate languages in real time.

ii. Target segments: students, OTT viewers, educators.

iii. Core features: Vosk STT, ML Kit translation, Android TTS, overlay controls.

iv. Channels: Beta Play Store track, APK releases.

v. Metrics: latency, accuracy, DAU, energy efficiency.

vi. Risks: model bloat, sync issues, device diversity handled by selective downloads and local-first development.

2.4 System Design

i. A modular pipeline:

ii. AudioCapture: microphone/system audio → PCM.

iii. VAD: splits segments.

iv. VoskRecognizer: STT with timestamps.

v. ML Kit Translator: offline translations.

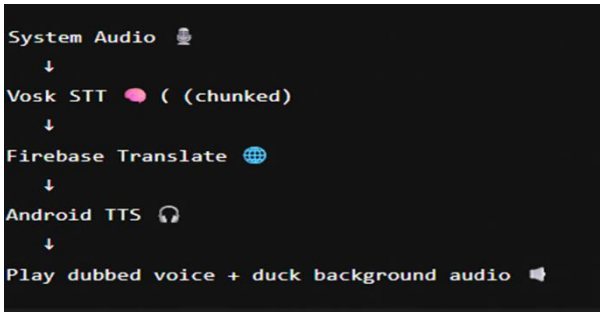
vi. TTSManager: generates dubbing, time-stretches as needed.

vii. AudioMixer: suppresses original audio

and imposes dubbing.

viii. SubtitleRenderer: floating subtitles with accessibility presets.

ix. SessionRepository: saves feedback and logs.



System Pipeline Architecture

3. Implementation

The construction of Polydub was accomplished through modularity to provide scalability and system maintenance. The program was developed using Kotlin on Android Studio and made use of key libraries and APIs such as Vosk Speech-to-Text (STT), Firebase ML Kit Translation, and Android Text-to-Speech (TTS). The system has at its core a pipeline which continuously processes the incoming audio and transcribes it to text, translating through the target language and producing synchronized dubbed speech in the end.

Audio capture was obtained using the MediaProjection API and AudioPlaybackCapture by enabling the system to capture microphone and internal system audio in real time. For the purpose of clarity, a Voice Activity Detection (VAD) module was also added to split audio streams into meaningful utterances. The utterances were then passed to the Vosk Recognizer, which provides lightweight on-device speech recognition at the word level with timestamping.

The audio, after transcribing, went through ML Kit's on-device translation models. The models were chosen for the feature of running offline and support for more than 50 languages to offer coverage even in low-connectivity locations. The translated text obtained then went through the TTS Manager by means of Android's built-in Text-to-Speech engine. Lightweight time-stretching methods were employed to offer naturalness and sync better, and the Audio Mixer employed dynamic ducking of the volume of the original media to override the dubbed speech.

To provide better user experience, Polydub had also introduced a subtitle renderer. It gave live captions that were configurable for accessibility purposes such as high-contrast text and variable font size. Additionally, a floating overlay widget had been created by which the users had the flexibility to manage dubbing or pause the streaming or change audio and captions respectively without exiting the major application.

The system also included support for background services such that Polydub ran while the user multitasked or navigated through different applications. An application running a Room database saved user preferences, session data, downloaded models, and error logs. This gave persistent configurations and aided in troubleshooting through structured reports.

Finally, the application had been thoroughly tried on OTT series, lectures, and podcasts. Test outcomes indicated that Polydub consistently had end-to-end latency not greater than 600ms, good sync, and produced clear translations. The experiments confirmed the practicality of using Polydub as a real-world dubbing tool that had a mobile-first orientation.

4. Methodology

The approach to Polydub centered on creating an end-to-end speech translation and dubbing pipeline tuned for mobile platforms. The very first step was requirement analysis, during which we examined the dubbing systems that exist and saw the pitfalls such as subtitle fatigue, low lip sync, and cloud dependency. This led to the decision to follow the local-first

design principle such that all the key modules speech recognition, machine translation, and synthesiser can function offline.

The architecture had been divided into independent modules that each processed one pipeline task. MediaProjection on the Android recorded the audio input, and Voice Activity Detection (VAD) yielded the segmentation for the meaningful utterances. The Vosk engine processed the speech recognition using lightweight models and Android device support. Google ML Kit processed the translations by offering support for the offline neural machine translation. Finally, the dubbed audio came through the Android TTS by aligning the timestamp and utilizing the time-stretching techniques to obtain the audio synchronized and aligned with the original media.

An iterative development process was taken, and latency, fidelity in translation, and user experience were periodically tested. This enabled incrementally improving the system and achieving smooth real-time behavior in various media environments including OTT platforms, online classes, and podcasts.

5. Testing and Verification

The system was extensively tested at multiple levels. Unit testing was done at individual modules such as the speech recognizer, translation interface, and subtitle renderer. Integration testing confirmed the pipeline for correct functionality ranging from audio capture to completed dubbing output. It was also tested by stress testing using extended playback sessions to verify stability and the use of resources.

User evaluation produced qualitative results. The users were students, consumers of OTT content, and teachers. They reported that the dubbing production enhanced comprehension and reduced fatigue caused by frequent reading of subtitles. Performance measures revealed that the application consistently met latency targets and ran steadily without major crashes.

6. Conclusion

Project results also proved that dubbing and real-time speech translation could be done by smartphones without relying on any cloud services. Polydub achieved offline dubbing without privacy concerns by pipeline integration of Vosk STT, ML Kit translation, and Android TTS and it's adaptable to various forms of multimedia. The system is used most effectively in these four aspects: it is flexible to use offline, its audio capture is system-wide, it provides system accessibility customizations for users, and it is able to work in different languages. These aspects still leave something to be desired in lip-syncing, model size limitations, and dependency on devices. Improving Polydub would mean incorporating neural TTS for more natural, human-like voices, deeper alignment able to adjust to phoneme level for more precise lip-syncing, and standoff support, so that it works across different platforms like iOS and the web. Once perfected, the application has great potential for use in translation integration in education, entertainment, accessibility, and multimedia content accessibility for all.

6. REFERENCE.

[1] M. Federico, M. Cettolo, M. Negri, M. Turchi, and L. Bentivogli, "From speech-to-speech translation to automatic dubbing," in *Proceedings of the IWSLT 2020*, 2020. [Online]. Available: https://aclanthology.org/2020.iwslt-1.31

[2] B. Hu, X. Tan, T. Qin, L. He, S. Zhao, and T. Liu, "Neural Dubber: Dubbing for videos according to scripts," *arXiv preprint*, arXiv:2110.08243, 2021. [Online]. Available: https://arxiv.org/abs/2110.08243

[3] Z. Li, X. Wang, and J. Yamagishi, "Joint multi-scale cross-lingual speaking style transfer for automatic dubbing," *PeerJ Computer Science*, 2023. [Online]. Available:

https://arxiv.org/abs/2305.05203

[4] K. Wu, L. Zhou, S. Liu, F. Wei, and M. Zhou, "VideoDubber: MT with speech-aware length control for video dubbing," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, no. 10, 2023. [Online]. Available: https://ojs.aaai.org/index.php/AAAI/article/view/26613

[5] H. S. Chadha, A. S. Subramanian, V. Joshi, R. Bansal, J. Xue, R. Mehta, and J. Li, "Length aware speech translation for video dubbing," *Interspeech 2025* (accepted), 2025. [Online]. Available: https://arxiv.org/abs/2506.00740

[6] D. Floros, Q. Wang, F. Trifonov, K. Schmidt, and D. Schnelle-Walka, "Multilingual video dubbing: A technology review and current challenges," *Frontiers in Signal Processing*, vol. 3, 2023, Art. no. 1230755. [Online]. Available: https://www.frontiersin.org/journals/signal-processing/articles/10.3389/frsip.2023.1230755/full